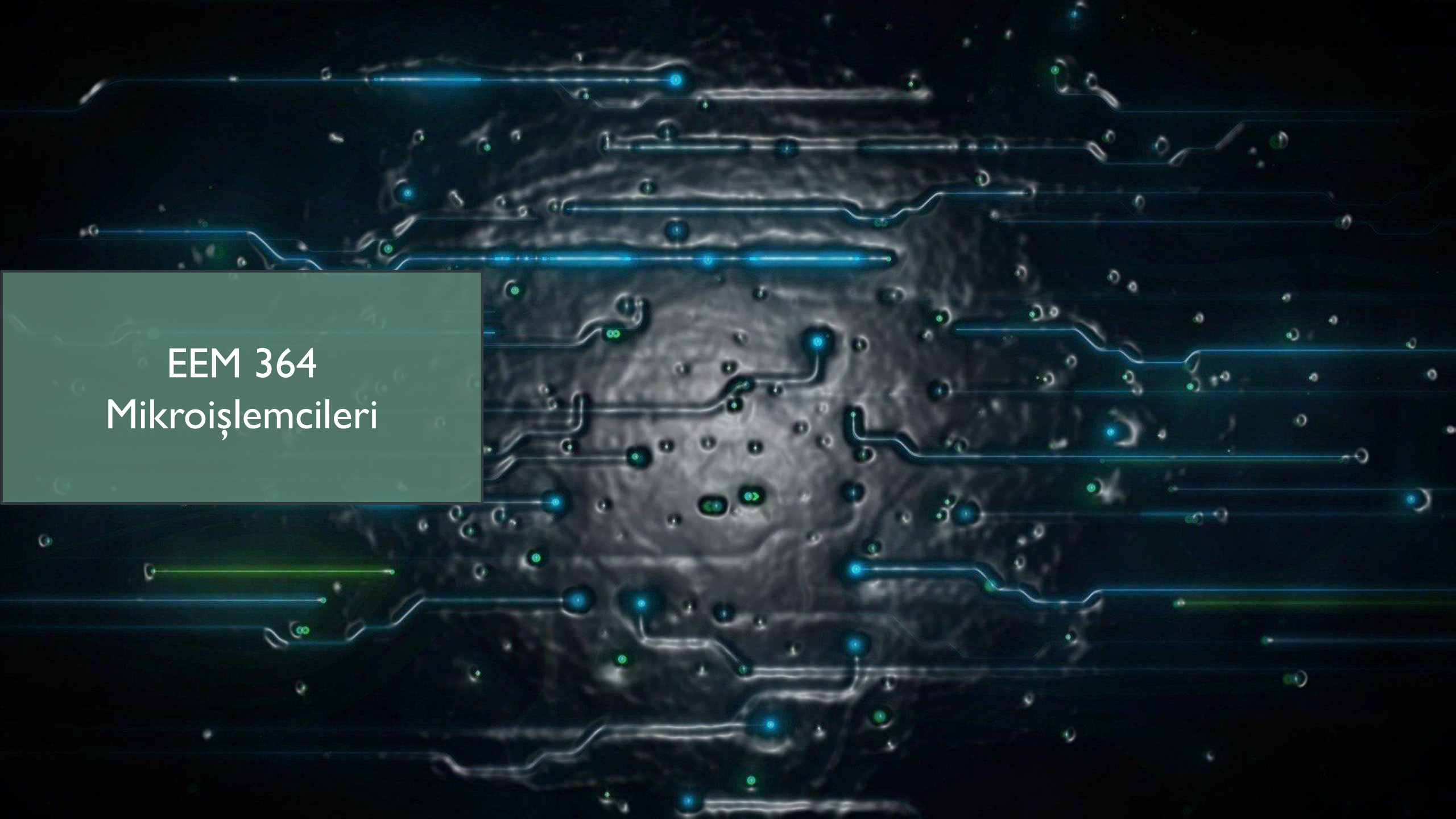

EEM 364
Mikroişlemcileri

```
.MODEL SMALL

.DATA

x   db 1,3,5,2,2,1,8,0,4, 7
lx equ $-x-1

.CODE

main proc far
    mov ax , @data
    mov ds , ax

L0:
    mov ax , 0
    mov cx , lx
    mov si , 0
    mov dl , 0


L:
    mov al , x[si]
    mov ah , x[si+1]

    call SWP
    inc si
    loop L

    cmp dl , 0

    jnz L0

HLT

main endp

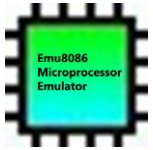
SWP proc

    cmp al , ah
    jle not_swp
    mov x[si+1] , al
    mov x[si]   , ah

    inc dl

    not_swp:
    ret
SWP endp
```

Sıralama algoritması



Çarpma ve Bölme

```
.MODEL SMALL

.DATA

A equ  15
C equ  4

Z dw 0

.CODE

main proc far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov bx , 0

    call carp

    mov Z , bx

    HLT

main endp


carp proc

    mov al , A
    mov cx , C


    CAR: add bx , ax
         loop CAR


    ret

carp endp
```

```
.MODEL SMALL

.DATA

X equ  53
B equ  7

Z dw 0

.CODE

main proc far

    mov ax , @data
    mov ds , ax

    mov ax , 0

    call bol

    hlt

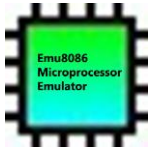
main endp


bol proc

    mov al , X
    mov bx , 0

    L:   inc ah
         sub al , B
         cmp al , B
         jge L
    ret

bol endp
```



- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

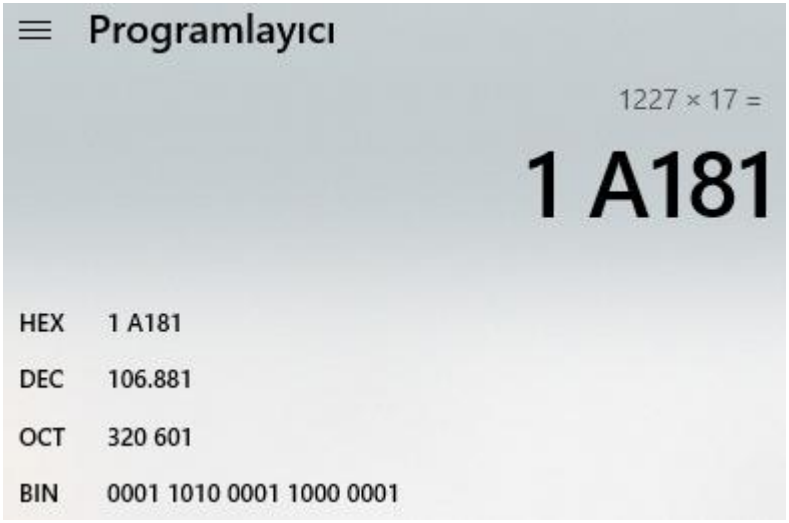
- CMP
- JE , JG , JL
- TEST
- PROC
- LOOP
- MUL

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

- **MUL ve IMUL**
- reg , reg
- reg , mem

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```



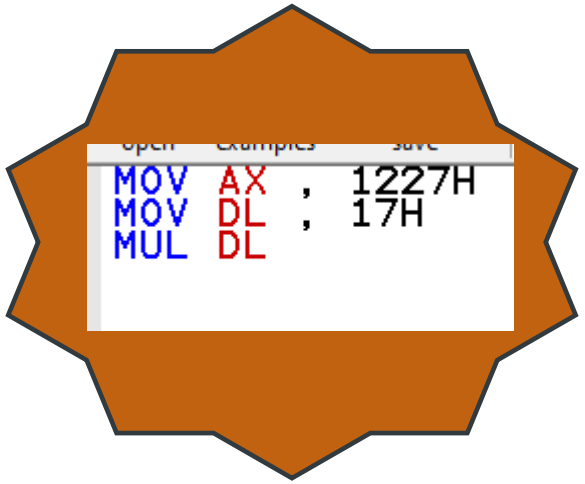
- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

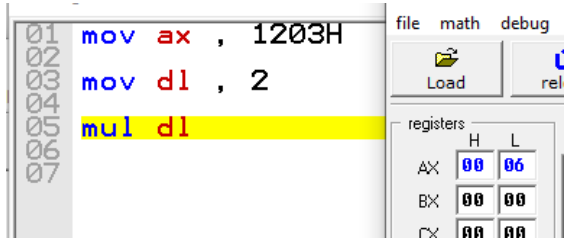
YAZMAÇLAR	
AH: 12	AL : 27
AX :A1 81	
DX: 00 01	

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
MOV AX , 1227H
MOV DX , 17H
MUL DX
```



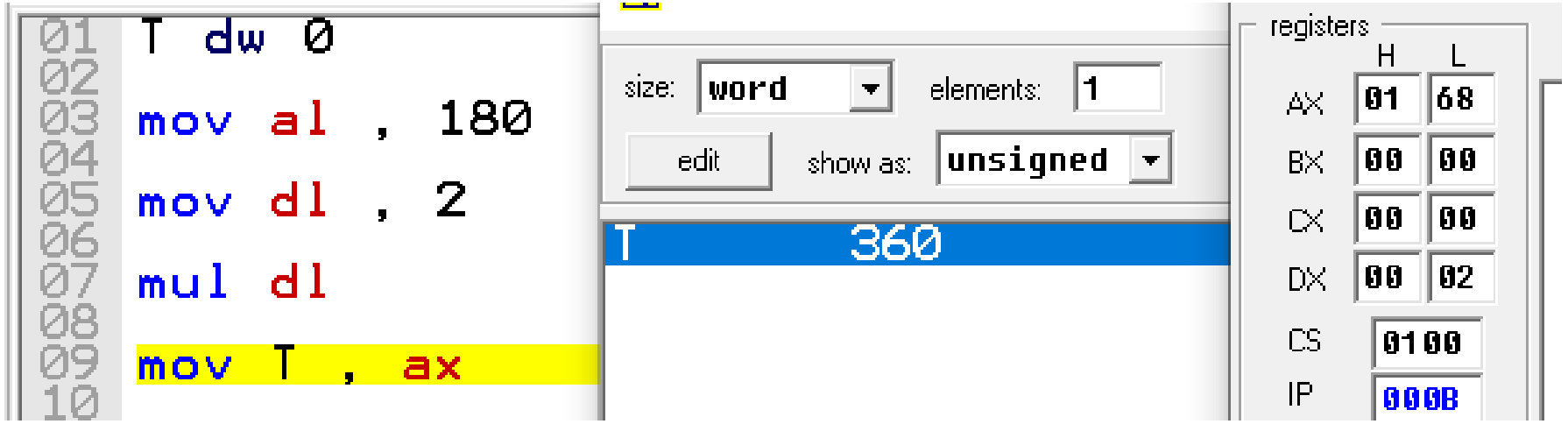




YAZMAÇLAR	
AH: 12	AL : 27
AX :A1 81	
DX: 00 01	

- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

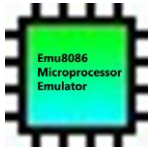


- **MUL ve IMUL**
- reg , reg
- reg , mem

- **Çarpılan**:
  - AL veya AX
- **Çarpan**:
  - BL,CL,DL
  - BX,CX,DX

```
01 .MODEL SMALL
02
03 .DATA
04
05 X equ  6
06
07 S dw 0
08
09
10 .CODE
11
12 mov ax , @data
13 mov ds , ax
14
15
16 mov ax , 0001H
17 mov cx , X
18
19 L:
20 mul cx
21 loop L
22
23 mov S , ax
24
25
26 HLT
27
```

Faktöriyel !



- **DIV ve IDIV**
- reg , reg
- reg , mem

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 27
AX: 10 01	

- **DIV ve IDIV**
- reg , reg
- reg , mem

- **Bölünen**:
  - AL veya AX
- **Bölen**:
  - BL, CL, DL
  - BX, CX, DX

```
MOV AL , 27H ; 39
MOV DL , 17H ; 23
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 27
AX: 10 01	

```
MOV AL , 42H; 66
MOV DL , 12H; 18
DIV DL
```

YAZMAÇLAR	
AH: ~	AL : 42
AX: 0C 03	

- JNZ/JZ
- CMP
- INC
- DEC
- OFFSET
- EQU
- LEA

- CMP
- JE , JG , JL , JNS
- TEST
- PROC
- LOOP
- MUL
- DIV
- PUSH
- POP

Asal Sayı Bulma

```
.MODEL SMALL

.DATA
x equ 53
y equ x/2

BN db  0
BM db 0
FL db 1

.CODE

main proc far

mov ax , @Data
mov ds , ax

mov cl , 2
mov dx , x

call asal


hlt
main endp

asal proc

    L:
    mov ax , dx
    div cl
    cmp ah , 0
    jz not_asal
    inc cl
    cmp cl , y
    jnz L


    D:

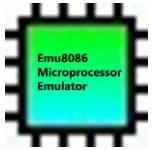
    ret

    not_asal:

    mov BN , cl
    mov BM , al
    mov FL , 0

    jmp D

asal endp
```



```
.model small

.data

x    db 2,5,3,4,8,1,2,10
lx equ $-x
MX   db 0

.code
 mov ax , @data
 mov ds , ax

 mov ax , 0
 mov si , 0

 call ort

HLT

ort proc

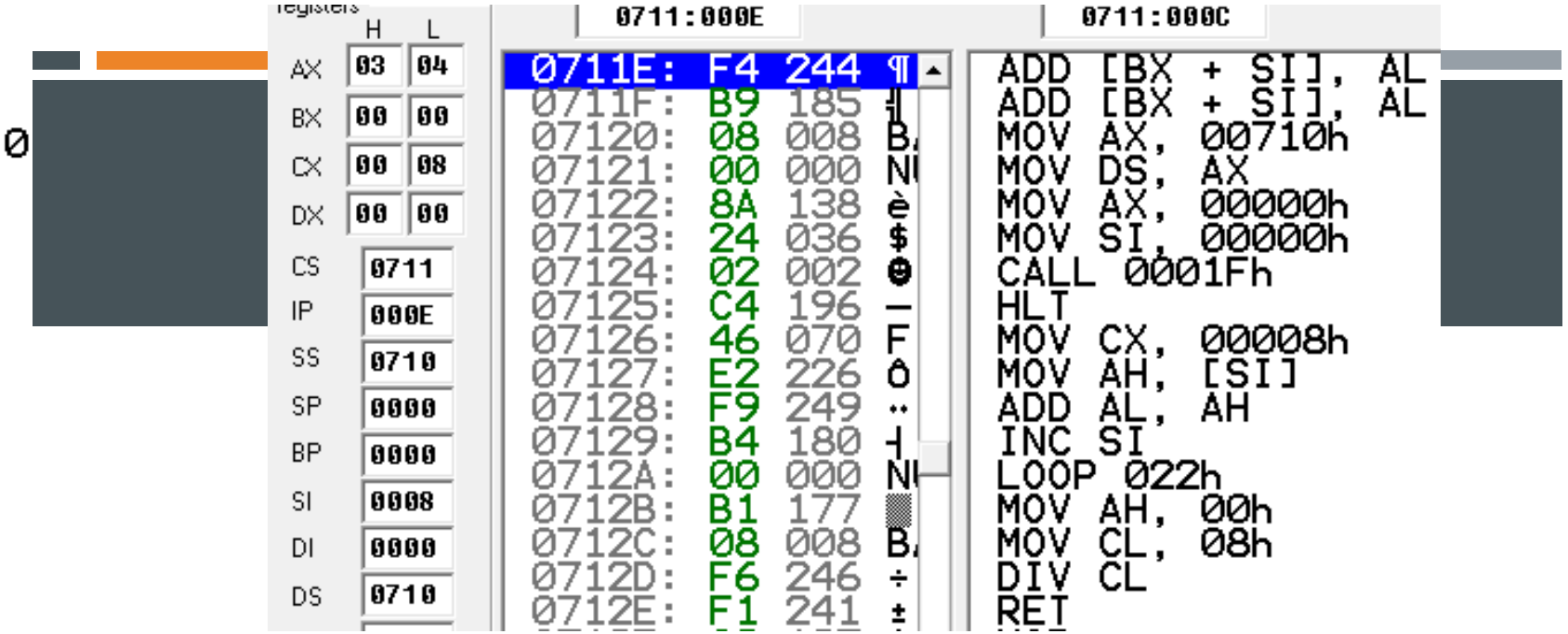
     mov cx , lx

     L:
          mov ah , x[si]
          add al , ah
          inc si

     loop L

     mov ah , 0
     mov cl , lx
     div cl

     ret
ort endp
```



İç İçe Döngüler

Belli bir sayıyı sayma

```
.MODEL SMALL

.DATA

 x  db 1,2,3,4 , 5,6,7,4 , 9,4,11,12
 lx equ 3
 ly equ 4

 S db 0

.CODE

Main PROC Far

    mov ax , @data
    mov ds , ax

    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly

        L2:

            mov al , x[si]
            inc si
            cmp al , 4
            je cnt
            D:

        loop L2

        pop cx

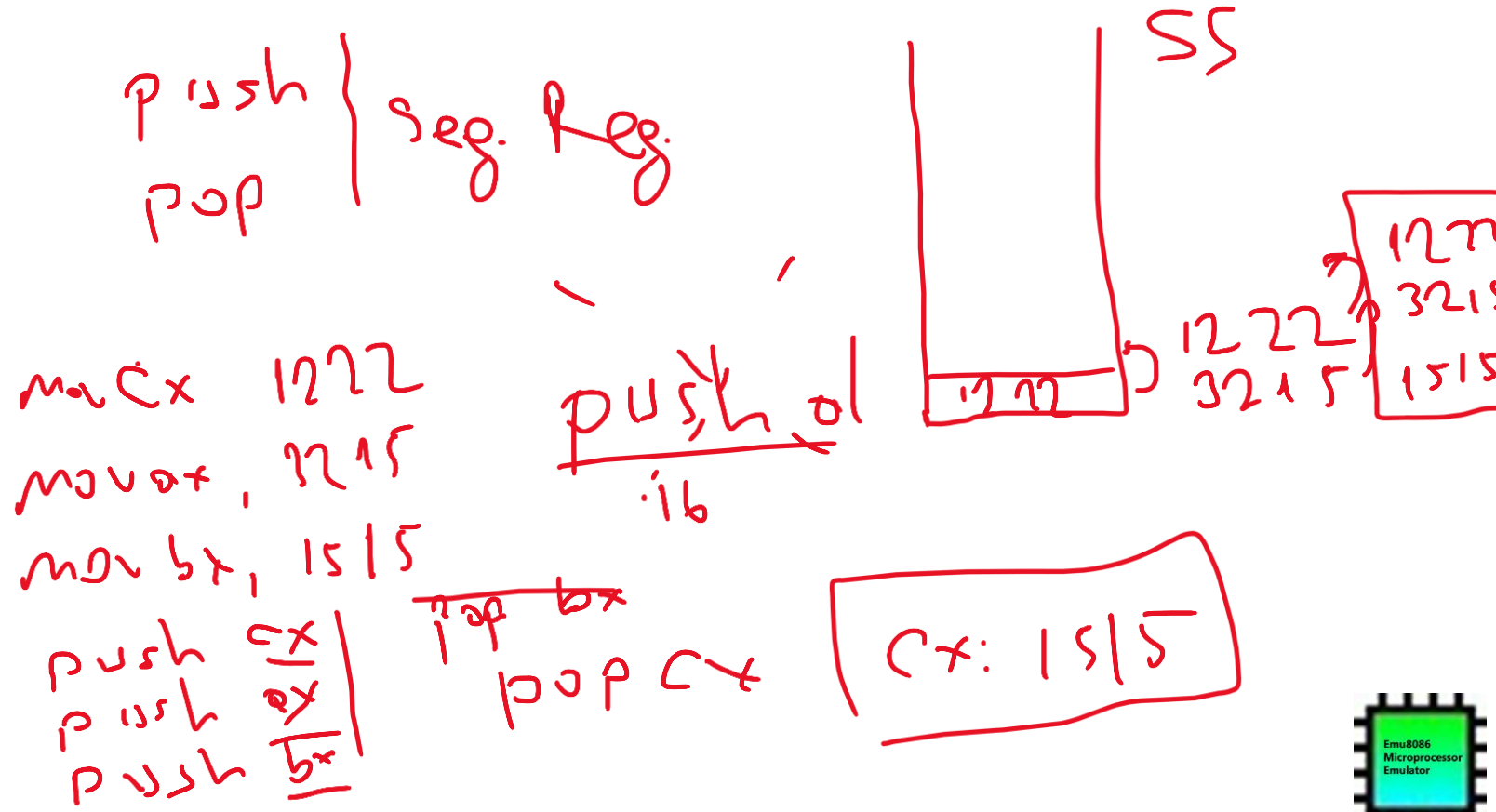
    loop L1

    mov S , dl

HLT

Main ENDP

cnt:
inc dl
jmp D
```



```
.MODEL SMALL

.DATA

 x  db 1,2,3,2 , 3,3,2,1 , 2,1,3,2 , 2,1,3,2
 lx equ 4
 ly equ 4

.CODE


mov ax , @data
mov ds , ax

mov ax , 0

mov si , 0

mov ch , 0


L1:
   mov cl , 0

   mov ax , 0

   L2:

      mov al , x[si]
      inc si
      cmp ch , cl
      je TPL

      D:

      inc cl
      cmp cl , ly
      jne L2

   inc ch
   cmp ch , lx
   jne L1


HLT

TPL:

add dh , al

jmp D
```



Diyagonal terimleri toplama

$i=j \sum m[i,j]$

Emu8086
Microprocessor
Emulator

```
.MODEL SMALL

.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)

.CODE

Main PROC Far

    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

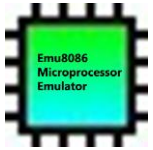
        pop cx

    loop L1


HLT

Main ENDP
```

Matris Satır Toplamı



```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
```

İç İçe Döngüler

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far
    mov ax , @data
    mov ds , ax


HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far
    mov ax , @data
    mov ds , ax

        mov al , x[si]
        add dl , al

HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far
    mov ax , @data
    mov ds , ax
```

```
        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2
```

```
HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)
.CODE
Main PROC Far
    mov ax , @data
    mov ds , ax


        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di


HLT
Main ENDP
```

İç İçe Döngüler

```
.MODEL SMALL
.DATA

 x  db 1,2,3,4 , 5,6,7,8 , 9,10,11,12
 lx equ 3
 ly equ 4

 T db lx dup(0)

.CODE
Main PROC Far
    mov ax , @data
    mov ds , ax


    mov ax , 0
    mov cx , lx

    mov si , 0
    mov di , 0

    L1: push cx
        mov cx , ly
        mov dl , 0

        L2:
            mov al , x[si]
            add dl , al
            inc si
        loop L2

        mov T[di] , dl
        inc di

        pop cx

    loop L1

HLT
Main ENDP
```

```
.MODEL SMALL

.DATA

 x  db 1,2,3,4,  5,6,7,8,  9,10,11,12
 lx equ $-x

.CODE

Main PROC FAR

     mov ax , @data
     mov ds , ax


     mov dh , 0

     L1:


        mov dl , 0

        L2:
            mov al , x[si]
            cmp x[si] , 7
            je K

            inc si


            inc dl
            cmp dl , 4
            jne L2


       inc dh
       cmp dh , 3
       jne L1

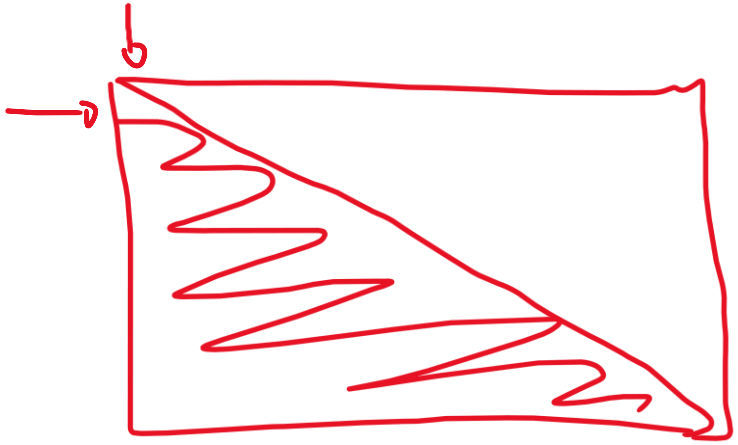

     K:
     hlt

Main ENDP
```

Matris içindeki bir elemanın indisini bulma

İç İçe Döngüler

Alt Üçgen Toplam



```
.model small

.data
X db 1 ,2 ,4, 3   ,    6 , 5 , 4 , 7   ,    3 , 6 , 7 , 9   ,   3 , 2 , 4 , 5

w db 1, 1, 1, 1

lx equ 4
ly equ 4

T db 0 dup(lx)

.code
main proc far

    mov ax , @data
    mov ds , ax


  mov cx , lx

  mov si , 0
  L1:

    push cx

    mov cx , ly
    mov di , 0
    mov ax , 0
    mov dx , 0

    L2:
        mov al , X[si]
        mov dl , w[di]

        mul dl

        add dh , al

        inc si
        inc di

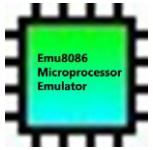
    loop L2

    mov T[bx] , dh
    inc bx
    pop cx

  loop L1


main endp
```

Matris Çarpması



S4 (P.30)) 4000 uzunluğunda bir genetik dizinin ilk 1000'ni göze, ikinci 1000 tanesi karaciğere üçüncü 1000 tanesi akciğere ve dördüncü 1000 tanesi ise kemiğe aittir. Bu genetik dizinin Adenin ("A"), Timin ("T"), Guanin ("G"), Sitozin ("S") nükleotit bazların baş harflerinden oluştuğu bilinmektedir.

XYZ isminde hayali bir kalıtısal hastalığın, "AATG" dizisinin 25'in üzerinde olmasıyla ortaya çıktığı bilindiğine göre her organ için bu rahatsızlığın ortaya çıkıp çıkmayacağını bulan kodun nasıl çalışacağını açıklayınız (**p.5**). ASM dilinde kodu yazınız. *H db 0,0,0,0* şeklindeki bir sembolünü ilgili organ için bayrak (0,1) için kullanınız. (**p.25**)

```
.MODEL SMALL

.DATA
    X DB 'ATGSATGAGATGTGATGTTAGTGTA'
    N EQU $-x
    S DB 0

.CODE

START:
    MOV AX, @DATA
    MOV DS, AX

    XOR SI, SI
    XOR DI, DI
    XOR AX, AX


L:
    MOV AL, X[SI]
    CMP AL, "A"
    JNE D

    INC SI
    MOV AL, X[SI]
    CMP AL, "T"
    JNE D

    INC SI
    MOV AL, X[SI]
    CMP AL, "G"
    JNE D

    INC S


    D:

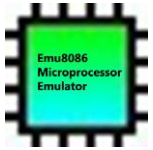
    INC SI

    CMP SI , N
    JL L


    HLT
```







```
.MODEL SMALL

.DATA
    X   DB 'ATGSATGSAATGSGTGSATGATGSG'   ; satir 0
        DB 'TGSATGTTGASTGGTGSGTGAASGA'   ; satir 1
        DB 'AATGSGTGAATGATGSATGATGAST'   ; satir 2
    T   DB 3 DUP(0)

.CODE

START:
    MOV AX, @DATA
    MOV DS, AX

    XOR DI, DI          ; DI = satir indeksi (0, 1, 2)
    XOR SI, SI          ; SI = dizi indeksi

SATIR_LOOP:
    XOR BX, BX
    MOV CX, 24

SATIR_ICI:
    MOV AL, X[SI]
    MOV AH, X[SI+1]
    MOV BL, X[SI+2]

    CMP AL, 'A'
    JNE ATLA
    CMP AH, 'T'
    JNE ATLA
    CMP BL, 'G'
    JNE ATLA

    INC BH              ; eger "ATG" eslesti ise

ATLA:
    INC SI
    LOOP SATIR_ICI

    MOV T[DI], BH       ; sayaci T[DI]'ye yaz


    INC DI
    CMP DI, 3
    JL SATIR_LOOP

    HLT
END START
```

